

프론트엔드 엔지니어 관점

Kiro Spec 모드를 제대로 쓰기 위한 시행착오로 배운 것들 (feat: FSD와 BDD)

최지연



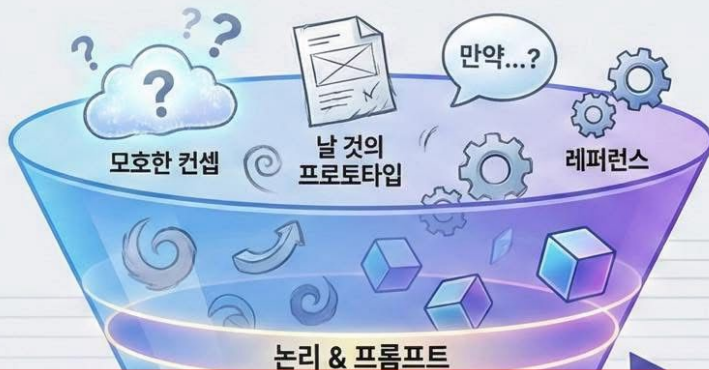


기존에서의 AI를 이용한 개발이 방향을 잃으면..



AI 시대 개발의 '스펙 깔때기'

1. 혼돈과 아이디어 (입력)



2. 구체화 및 정제 (필터)



AI 기반 속도:
1인 개발자 역량 강화

3. AI 가속 및 결과 (실행)



핵심: 아이디어가 논리적 스펙으로 정제되면, AI가 나머지를 즉시 실행합니다.

1. 속도에서의 생산성은 빨라졌다, 하지만 —

1. 속도에서의 생산성은 빨라졌다. 하지만 —



간단한 데모, 나만의 날 것을 나타내는데는 문제가 없다
하지만..

1. 속도에서의 생산성은 빨라졌다. 하지만 —



아니야 그게 아니고...

📷 🟢

1. 속도에서의 생산성은 빨라졌다. 하지만 —

Rule 파일도
제각각

문서가 기니까
안 읽음

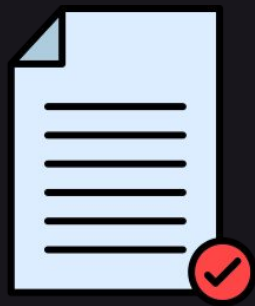
메신저, Docs에
문서가
여기저기
퍼져있음

이 문서가
최신인가..?

애매한 부분을
설명하기
어려우니까
계속 누락됨

외국어로도
써야해서
문서도 2배!

1. 속도에서의 생산성은 빨라졌다. 하지만 —



문서가 한곳에 모여있고



읽는 피로도가 적어야하고



규칙이 간단하지만 분명해야

Spec모드로 옮길 때 비용이 적게 드는 방법이 있을까?

2. BDD(Cucumber)를 통한 요구사항 정제

2. BDD(Cucumber)를 통한 요구사항 정제



Cucumber란?

BDD (Behavior-Driven Development) 프레임워크로,
비즈니스 요구사항을 자연어로 작성하고 이를 자동화된 테스트로
변환하는 도구예요.

1. 자연어 기반 (Gherkin)

```
# language: ko
기능: 학생 로그인
  학생이 이메일과 비밀번호로 로그인한다

시나리오: 성공적인 로그인
  조건 회원가입된 학생이 있다
  만약 로그인 페이지에 접속한다
  그리고 이메일 "student@example.com"을 입력한다
  그리고 비밀번호 "password123"을 입력한다
  그리고 로그인 버튼을 클릭한다
  그러면 학생 대시보드로 이동한다
```

2. 다국어 지원 (40개국 이상)

```
# language: ko
기능: 학생 회원가입

# language: ja
フィーチャ: 学生登録

# language: en
Feature: Student Registration
```

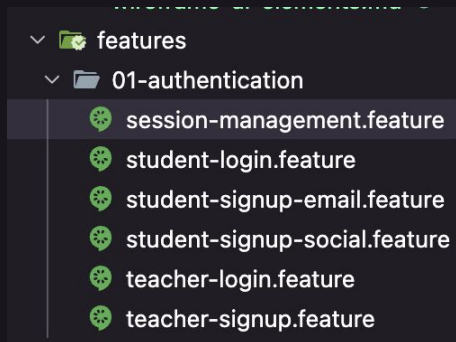
2.

3. 태그 시스템

```
# features/product/product-search.feature

@product @search @happy-path
시나리오: 상품 검색 성공
  조건 회원으로 로그인되어 있다
  만약 상품 목록 페이지에 접속한다
  그리고 "노트북"을 검색한다
  그러면 검색 결과가 표시된다
```

2. BDD(Cucumber)를 통한 요구사항 정제



```
15 # 로그인
16 # =====
17
18 @locale-kr @locale-jp @p1-auth @session @logout @happy-path
19 시나리오: 학생 로그인아웃
20   조건 학생으로 로그인되어 있다
21   그리고 학생 대시보드에 있다
22   만약 "로그아웃" 버튼을 클릭한다
23   그러면 로그인아웃에 성공한다
24   그리고 로그인 페이지로 이동한다
25   그리고 세션이 종료된다
```



```
1 [
2   {
3     "description": " 사용자의 로그인 세션을 자동으로 관리한다.\n 활동 중에는 자동으로 세션을 연장하고, 비활동 시 자동 로그인아웃.",
4     "elements": [
5       {
6         "description": "",
7         "id": "세션-관리;학생-로그아웃",
8         "keyword": "시나리오",
9         "line": 19,
10        "name": "학생 로그인아웃",
11        "steps": [
12          {
13            "keyword": "Before",
14            "hidden": true,
15            "result": [
```

2. BDD(Cucumber)를 통한 요구사항 정제

개발자랑 비개발자와 공통 언어(자연어)로 한곳에 모아놓고 관리가 가능.

HTML 리포트 쉽게 생성 가능, 그 다음 도식화를 했을때 이해도가 높아졌다.

기능별 시나리오로 슬라이스해 운영진 이해도 도모, 태그 기능이 있어 빠르게 검색 및 컨펌

KIRO Spec 모드의 requirements.md 문서와 닮아있음 — KIRO가 읽을 수 있는 빠른 토대 마련

2. BDD(Cucumber)를 통한 요구사항 정제

```
15 # 로그인
16 # =====
17
18 @locale-kr @locale-jp @p1-auth @session @logout @happy-path
19 시나리오: 학생 로그아웃
20   조건 학생으로 로그인되어 있다
21   그리고 학생 대시보드에 있다
22   만약 "로그아웃" 버튼을 클릭한다
23   그러면 로그아웃에 성공한다
24   그리고 로그인 페이지로 이동한다
25   그리고 세션이 종료된다
```

Requirement 1: Audio File

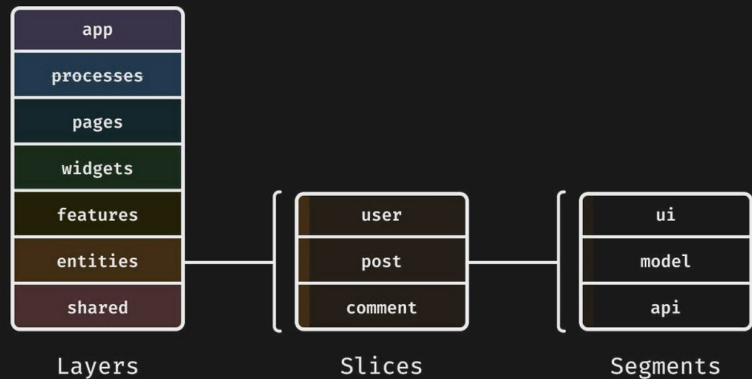
****User Story:**** As a user, I want to visualize their pitch contour

Acceptance Criteria

1. WHEN a user provides an audio file data using the native sample rate
2. WHEN an invalid file path is provided, the system SHALL return an error message
3. THE System SHALL support common audio formats

3. FSD, AI의 가이드레일이 되다

3. FSD, AI의 가이드레일이 되다



```
src/  
├── app/  
├── entities/  
├── features/  
├── shared/  
└── widgets/
```

✓ FSD의 app layer
✓ FSD의 entities layer
✓ FSD의 features layer
✓ FSD의 shared layer
✓ FSD의 widgets layer

FSD를 차용한 구조

FSD란?

FSD(Feature-Sliced Design) 프론트엔드 애플리케이션 개발에서 코드를 '기능(Feature)' 중심으로 구조화하는 최신 아키텍처 방법론

3. FSD, AI의 가이드레일이 되다

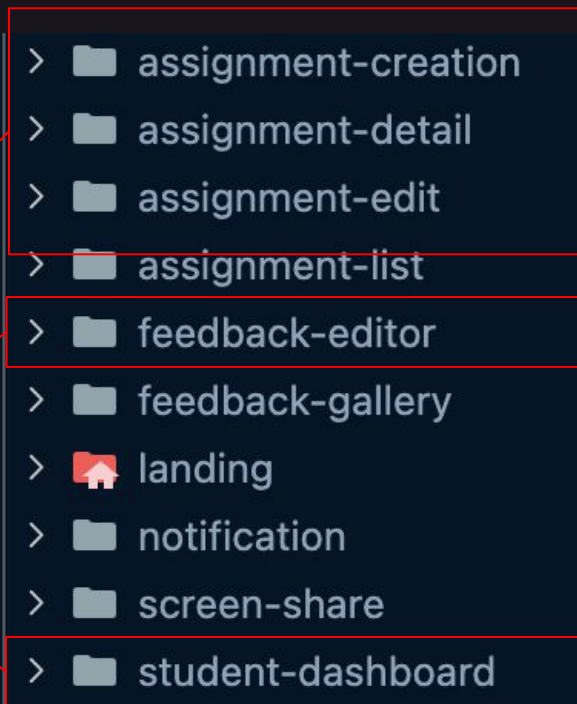
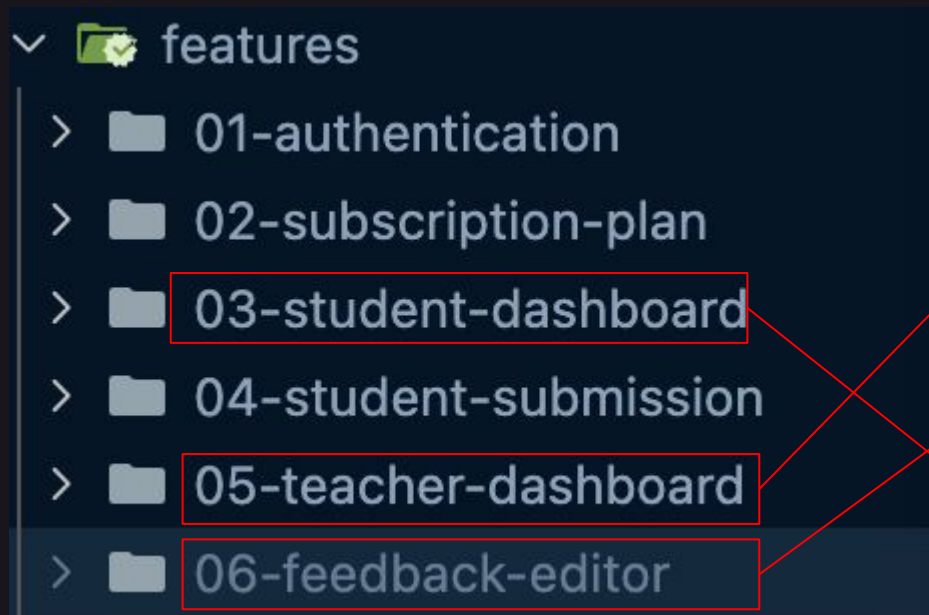
```
src/  
├── app/  
├── components/  
│   ├── AssignmentForm.tsx  
│   ├── AssignmentList.tsx  
│   ├── StudentCard.tsx  
│   ├── TeacherDashboard.tsx  
│   └── ... (100+ 파일)  
├── hooks/  
└── lib/
```

기본 Next.js 구조

```
src/  
├── entities/assignment/  
├── features/  
│   ├── assignment-creation/  
│   ├── assignment-list/  
│   └── submission/  
└── shared/
```

FSD를 차용한 구조

3. FSD, AI의 가이드레일이 되다



3. FSD, AI의 가이드레일이 되다

기본 Next.js 구조에서의 경우 



3. FSD, AI의 가이드레일이 되다

현재 FSD 구조의 경우

NEXT..

1단계: 파일 위치 찾기
(5-10초)

AI: "과제 제출? →
features/submission 폴더"

→ 즉시 위치 파악
→ 불확실성: **10%**

2단계: 의존성 파악
(10-20초)

AI: "submission
폴더만 보면 됨"

→ 명확한 경계
→ entities/
assignment만
확인
→ 영향 범위 명확

3단계: 코드 작성
(1-2분)

AI: "패턴이
명확함"

→ model/
useSubmission.ts
→ ui/Submission
Form.tsx
→ 일관된 구조

4단계: 검증
(10-20초)

AI: "features 간
독립적"

→ 최소한의 검증
→ 사이드 이펙트 **거의
없음**

총 소요 시간: **1.5분 - 3분**

정확도: **85-95%**

재작업 확률: **10%**

3. FSD, AI의 가이드레일이 되다

협업 효율성 비교

협업 시나리오	기본 구조	FSD 구조	개선율
파일 충돌	60%	5%	92% 감소
의도 파악 시간	3-5분	10-20초	85% 단축
재작업 확률	50%	10%	80% 감소
스타일 일치도	30%	90%	+60%p
병렬 작업 가능성	낮음	높음	3배 향상

전체 효율성 매트릭스

Kiro 단독 작업

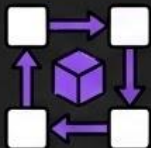
구조	평균 작업 시간	정확도	재작업률
기본 Next.js	6-8분	65%	40%
FSD (82점)	1.5-2.5분	90%	10%
FSD + Cucumber	1-2분	97%	3%

개선율: 기본 대비 **83%** 향상

다른 AI와 협업

구조	의도 파악	충돌률	재작업률
기본 Next.js	3-5분	60%	50%
FSD (82점)	10-20초	5%	10%
FSD + Cucumber	5-10초	1%	3%

4. 결론 - 강력한 궁합 이유



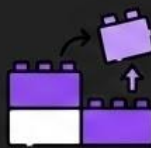
구조적 일치: Requirements의 AC가 이미
Given-When-Then 형식



자동화 가능: AI가 Spec → Feature 변환
가능



추적성: Requirements → Properties →
Tasks → Features 완벽 연결



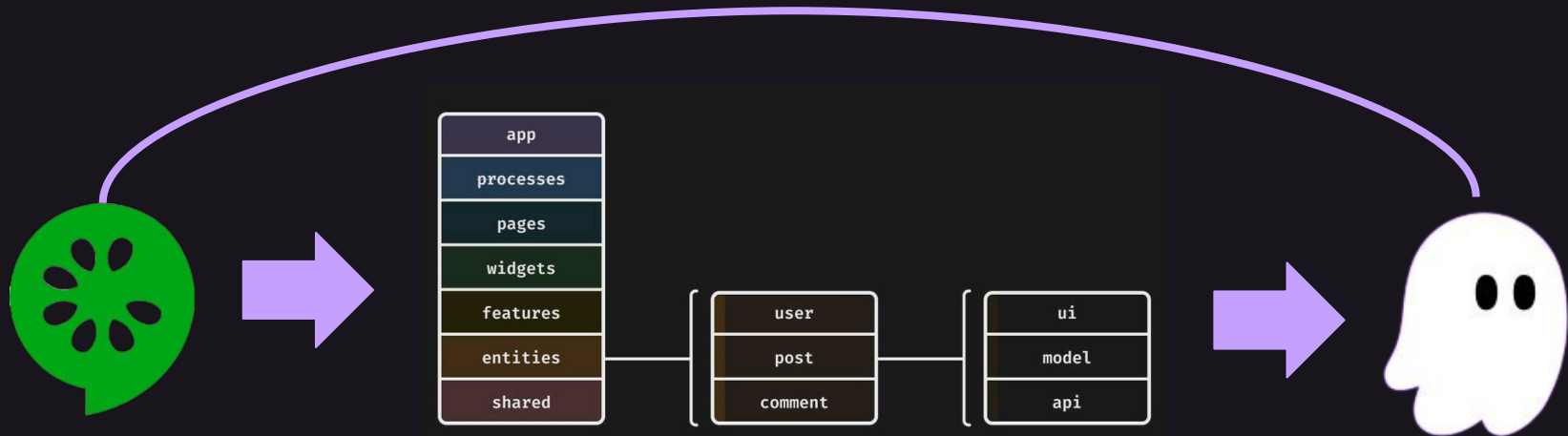
점진적 도입: 기존 Spec 구조 유지하며
Feature만 추가



팀 협업: 비개발자도 Feature 파일로
Feature 검증 가능

KEYWORD

반복 및 개선



시나리오

가이드레일

점진적 문서화 및
개발

KIRO Spec 모드 성능 극대화 측정 지표



지표명	측정 기준 및 근거
아키텍처 규칙 위반율	FSD(Feature-Sliced Design) 계층 간 의존성 규칙 준수 여부 조사. 전체 492개 파일 중 위반 사례 1건 미만 (의존성 그래프 분석 기준)
히스토리 추적 소요 시간	기존 레거시 코드 파악(평균 15분) 대비, BDD 시나리오와 Spec 문서만으로 전체 맥락을 파악하는 데 소요된 시간 비교 측정
평균 이터레이션 횟수	최초 Spec 입력 후 '추가 수정 요청' 없이 비즈니스 로직이 완성된 비율. 단순 프롬프트 방식 대비 생산성 3.7배 향상 확인

정리

1. Cucumber 시나리오: 무엇을 만들지 명확히



2. FSD 구조: 어디에 만들지 명확히



3. 의존성 명시: 무엇을 사용할지 명확히



3. BDD는 AI의 “검증 도구”다

Cucumber 시나리오를 작성하면:

- ✓ 요구사항이 명확해짐
- ✓ Kiro가 테스트 코드 자동 생성
- ✓ 엣지 케이스 사전 발견

2. FSD는 AI의 “가이드레일”이다

FSD 구조를 Spec에 명시하면:

- ✓ Kiro가 올바른 위치에 코드 생성
- ✓ 의존성 규칙 자동 준수
- ✓ 폴더 구조 일관성 유지

감사합니다!